

File Compression Using Huffman Coding in C

1. Introduction to File Compression

File compression is the process of reducing the size of data so that it occupies less storage space and can be transmitted faster over networks.

Compression is widely used in:

- ZIP files
- Image formats
- Audio formats
- Video streaming
- Databases
- Operating systems
- Networking systems

The major goal is:

- Reduce storage requirements
- Reduce transmission time
- Improve efficiency

2. Types of Compression

Compression techniques are divided into two categories:

2.1 Lossless Compression

Lossless compression allows the original data to be reconstructed exactly after decompression.

No information is lost.

Examples

- Huffman Coding
- Run-Length Encoding (RLE)
- Lempel-Ziv-Welch (LZW)

- DEFLATE (ZIP)

Applications

- Text files
- Source code
- Executable files
- Medical records
- Databases

2.2 Lossy Compression

Lossy compression removes some information permanently to achieve higher compression ratios.

Examples

- JPEG
- MP3
- MPEG

Applications

- Images
- Audio
- Video

3. What is Huffman Coding?

Huffman Coding is a:

- Greedy algorithm
- Lossless compression technique
- Variable-length encoding scheme

The key idea:

Frequently occurring characters get shorter binary codes, while infrequent characters get longer binary codes.

This reduces the total number of bits needed.

4. Fixed-Length vs Variable-Length Encoding

4.1 Fixed-Length Encoding

Suppose we have 8 characters.

Each character may require:

$$\log_2(8) = 3$$

So every character uses 3 bits.

Example:

Character	Code
A	000
B	001
C	010
D	011
E	100
F	101
G	110
H	111

Problem:

Even frequently used characters still use 3 bits.

4.2 Variable-Length Encoding

Frequently used characters receive shorter codes.

Example:

Character	Frequency	Code
A	45	0
B	13	101
C	12	100
D	16	111
E	9	1101
F	5	1100

Now the average bits per symbol become much smaller.

5. Prefix Codes

Huffman Coding uses **prefix codes**.

A prefix code means:

No code is a prefix of another code.

Character	Code
A	0
B	10
C	110

Valid because:

- 0 is not the prefix of 10
- 10 is not the prefix of 110

This allows unambiguous decoding.

6. Core Data Structures Used

Huffman Coding relies heavily on:

1. Binary Trees
2. Priority Queues
3. Min Heaps

7. Binary Tree Review

Each Huffman code is represented using a binary tree.

Structure of Huffman Tree

- Left edge $\rightarrow$ 0
- Right edge $\rightarrow$ 1
- Leaf nodes $\rightarrow$ characters
- Internal nodes $\rightarrow$ frequencies

Example node:

```

struct Node {
    char data;
    unsigned frequency;
    struct Node *left;
    struct Node *right;
};

```

8. Priority Queue

A priority queue removes elements according to priority.

In Huffman Coding:

- Smaller frequencies have higher priority.

Operations:

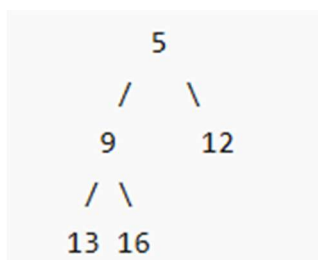
Operation	Purpose
Insert	Add node
Extract-Min	Remove smallest frequency
Heapify	Maintain heap

9. Min Heap

A Min Heap is a complete binary tree where:

Parent node $\leq$ child nodes

Example:



The smallest value is always at the root.

10. Why Min Heap in Huffman Coding?

Huffman repeatedly needs:

- Smallest frequency node
- Second smallest frequency node

Min Heap provides:

Operation	Complexity
Insert	$O(\log n)$
Extract-Min	$O(\log n)$

Without heaps, finding minimum repeatedly becomes inefficient.

11. Huffman Coding Algorithm

Steps:

1. Count character frequencies
2. Create leaf nodes
3. Insert all nodes into Min Heap
4. Remove two smallest nodes
5. Create new internal node
6. Insert combined node back
7. Repeat until one node remains

That remaining node becomes the root of the Huffman Tree.

12. Example Walkthrough

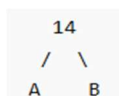
Suppose we have:

Character	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

Step 1: Pick Two Smallest

A(5) and B(9)

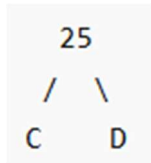
Create:



Step 2

Now frequencies become: 12, 13, 14, 16, and 45

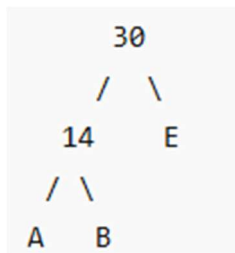
Pick C(12) and D(13):



Step 3

Now frequencies become: 14, 16, 25, and 45

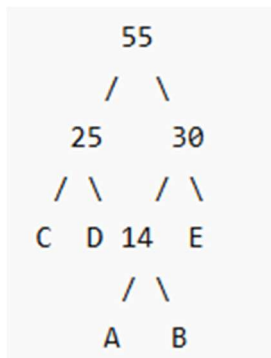
Pick 14 and 16:



Step 4

Now frequencies become: 25, 30 and 45

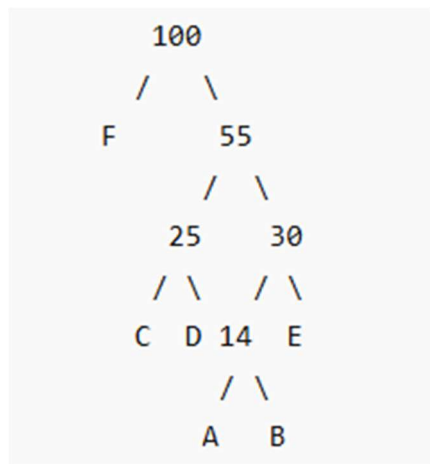
Pick 25 and 30:



Step 5

Now frequencies become: 45 and 55

Pick 45 and 55:



13. Generating Huffman Codes

Traverse:

- Left → append 0
- Right → append 1

Codes become:

Character	Code
F	0
C	100
D	101
A	1100
B	1101
E	111

14. Huffman Tree Node Implementation

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    unsigned freq;
    struct Node *left;
    struct Node *right;
};
```

15. Creating a Node

```
struct Node* createNode(char data, unsigned freq) {

    struct Node* node =
        (struct Node*) malloc(sizeof(struct Node));

    node->data = data;
    node->freq = freq;

    node->left = NULL;
    node->right = NULL;

    return node;
}
```

16. Min Heap Structure

```
struct MinHeap {
    unsigned size;
    unsigned capacity;
    struct Node** array;
};
```

17. Swapping Heap Nodes

```
void swapNode(struct Node** a, struct Node** b) {  
  
    struct Node* temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

18. Heapify Operation

Heapify restores heap order.

```
void minHeapify(struct MinHeap* heap, int index) {  
  
    int smallest = index;  
  
    int left = 2 * index + 1;  
    int right = 2 * index + 2;  
  
    if (left < heap->size &&  
        heap->array[left]->freq <  
        heap->array[smallest]->freq)  
  
        smallest = left;  
  
    if (right < heap->size &&  
        heap->array[right]->freq <  
        heap->array[smallest]->freq)  
  
        smallest = right;  
  
    if (smallest != index) {  
  
        swapNode(&heap->array[smallest],  
                &heap->array[index]);  
  
        minHeapify(heap, smallest);  
    }  
}
```

19. Extract Minimum

```
struct Node* extractMin(struct MinHeap* heap) {  
  
    struct Node* temp = heap->array[0];  
  
    heap->array[0] =  
        heap->array[heap->size - 1];  
  
    heap->size--;  
  
    minHeapify(heap, 0);  
  
    return temp;  
}
```

20. Insert into Min Heap

```
void insertMinHeap(struct MinHeap* heap,
                  struct Node* node) {

    heap->size++;

    int i = heap->size - 1;

    while (i &&
           node->freq <
           heap->array[(i - 1) / 2]->freq) {

        heap->array[i] =
            heap->array[(i - 1) / 2];

        i = (i - 1) / 2;
    }

    heap->array[i] = node;
}
```

21. Building Huffman Tree

```
struct Node* buildHuffmanTree(  
    char data[],  
    int freq[],  
    int size) {  
  
    struct Node *left, *right, *top;  
  
    struct MinHeap* heap =  
        createMinHeap(size);  
  
    for (int i = 0; i < size; i++)  
        insertMinHeap(heap,  
            createNode(data[i], freq[i]));  
  
    while (heap->size != 1) {  
  
        left = extractMin(heap);  
        right = extractMin(heap);  
  
        top = createNode('$',  
            left->freq + right->freq);  
  
        top->left = left;  
        top->right = right;  
  
        insertMinHeap(heap, top);  
    }  
  
    return extractMin(heap);  
}
```

22. Printing Huffman Codes

```
void printCodes(struct Node* root,
               int arr[],
               int top) {

    if (root->left) {

        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if (root->right) {

        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }

    if (!(root->left) && !(root->right)) {

        printf("%c: ", root->data);

        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);

        printf("\n");
    }
}
```

23. Complete Example

```
int main() {  
  
    char data[] = {'A', 'B', 'C', 'D', 'E', 'F'};  
    int freq[] = {5, 9, 12, 13, 16, 45};  
  
    int size = 6;  
  
    struct Node* root =  
        buildHuffmanTree(data, freq, size);  
  
    int arr[100], top = 0;  
  
    printCodes(root, arr, top);  
  
    return 0;  
}
```

24. Compression Example

Suppose text: AAAAABBC

Frequency table:

Character	Frequency
A	5
B	2
C	1

Possible Huffman Codes:

Character	Frequency
A	1
B	01
C	00

Encoded output: 11111010100

Original ASCII:

Each character: 8 bits

Original size: $8 \times 8 = 64$ bits

Compressed size: 11 bits

Huge reduction achieved.

25. Advantages of Huffman Coding

- Optimal prefix coding
- Lossless
- Efficient for repeated symbols
- Widely used
- Good compression ratio

26. Disadvantages

- Requires frequency table
- Tree must be reconstructed during decoding
- Not ideal when frequencies are uniform
- Dynamic data streams are harder

27. Real-World Applications

Huffman Coding is used in:

- ZIP compression
- PNG images
- JPEG entropy encoding
- MP3 encoding
- Network protocols

28. Encoding Process

1. Read file
2. Count frequencies
3. Build Huffman Tree
4. Generate codes
5. Replace characters with codes
6. Write compressed bitstream

29. Decoding Process

- Rebuild Huffman Tree
- Read bits one by one
- Traverse tree
- Reach leaf node
- Output character
- Restart from root

References:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.).

Topic	Chapter	Page
Binary Heaps	Chapter 6	pp. 151-191
Priority Queues	Chapter 6	pp. 163-176
Greedy Algorithms	Chapter 16	pp. 442-489
Huffman Coding	Section 16.3	pp. 470-479

2. Weiss, M. A. (1997). *Data structures and algorithm analysis in C* (2nd ed.).

Topic	Chapter	Page
Trees	Chapter 4	pp. 89-154
Binary Heaps	Chapter 6	pp. 233-273
Priority Queues	Chapter 6	pp.233-273
Algorithm Analysis	Chapter 2	pp. 13-55

3. Sedgewick, R. (1998). *Algorithms in C: Parts 1–4: Fundamentals, data structures, sorting, searching* (3rd ed.).

Topic	Chapter	Page
Priority Queues	Chapter 9	pp. 305 - 346
Heap Structures	Chapter 9	pp. 305 - 346
Trees	Chapter 12-14	pp. 445-604

4. Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.)

Topic	Chapter	Page
Structures	Chapter 6	pp. 127 – 158
Dynamic Memory Allocation	Chapter 8	pp. 167-185
Pointers	Chapter 5	pp. 93-126
File Handling	Chapter 8	pp. 174 - 179

5. Sayood, K. (2017). *Introduction to data compression* (5th ed.).

Topic	Chapter	Page
Entropy	Chapter 2	pp.21-66
Huffman Coding	Chapter 3	pp. 67-115
Adaptive Huffman Coding	Chapter 4	pp. 117-146

6. Salomon, D., & Motta, G. (2010). *Data compression: The complete reference* (5th ed.).

Topic	Chapter	Page
Huffman Coding & Prefix Codes	Chapter 2	pp. 29-78
Adaptive Huffman	Chapter 4	pp. 129-170
Arithmetic Coding	Chapter 5	pp. 171-214